

Fiche récapitulative – STA211

Application MICE : imputation multiple sur les données Diabète

10 juin 2026

Résumé

Cette fiche présente un cas pratique d'imputation multiple par équations chaînées (MICE) sur le jeu de données Diabète (Pima Indians Diabetes). Elle couvre l'analyse exploratoire du dispositif de manquant, la mise en œuvre de l'algorithme MICE, la vérification de la convergence, la comparaison des méthodes d'imputation, le pooling des modèles, et l'analyse de sensibilité.

Table des matières

1	Introduction	2
1.1	Chargement des packages	2
1.2	Importation	2
2	Analyse exploratoire	2
2.1	Liens entre variables	2
2.2	Analyse du dispositif de manquant	2
2.3	Nature du mécanisme de manquant	3
3	Imputation multiple avec MICE	3
3.1	Paramètres par défaut	3
3.2	Vérification de la convergence	3
3.3	Comparaison des distributions imputées vs observées	4
3.4	Analyse par marginmatrix	4
3.5	Overimputation	4
4	Choix des méthodes d'imputation	4
4.1	Méthodes disponibles pour MICE	4
4.2	Comparaison par overimputation	5
5	Analyse après imputation	5
5.1	Régression logistique sur chaque tableau imputé	5
5.2	Pooling des résultats (règles de Rubin)	5
5.3	Comparaison avec les cas-complets	6
6	Analyse de sensibilité	6
7	Résumé du pipeline	6
	Questions clés (QCM)	7

Introduction

L'étude de cas applique la méthode d'imputation multiple par équations chaînées (MICE) aux données **Diabète** (Pima Indians Diabetes Database). L'objectif est de traiter les valeurs manquantes avant une classification supervisée (prédiction du diabète).

Données : <https://github.com/niharikagulati/diabetesprediction>

Chargement des packages

```
1 library(mice)
2 library(micemd)
3 library(FactoMineR)
4 library(parallel)
5 library(DescTools)
6 library(VIM)
7 library(mvtnorm)
8 library(funModeling)
```

Importation

```
1 load("diabetes.Rdata")
2 dim(diabetes) # 768 x 9
```

Le jeu de données contient 768 observations et 9 variables : 8 prédicteurs (Preg, Glucose, BP, Skin.Thick, Insulin, BMI, DPF, Age) et la réponse binaire Outcome.

Analyse exploratoire

Liens entre variables

```
1 summary(diabetes)
2 pairs(diabetes[, 1:8], cex = .2)
3 matcor <- cor(diabetes[,1:8], use = "pairwise.complete.obs")
4 PlotCorr(matcor)
5
6 boxplot(diabetes[, "Age"] ~ diabetes$Outcome)
7 sapply(colnames(diabetes)[1:8],
8   FUN = function(xx, diabetes){
9     boxplot(diabetes[,xx] ~ diabetes$Outcome,
10       main = xx, ylab = xx, xlab = "outcome")
11   }, diabetes = diabetes)
```

Analyse du dispositif de manquant

Analyse univariée : on utilise `aggr` du package VIM pour visualiser la proportion de valeurs manquantes par variable.

```
1 res.aggr <- aggr(diabetes)
2 res.aggr$missings
3 cbind(res.aggr$tabcomb, res.aggr$percent)
```

Analyse bivariée : on utilise le V de Cramer entre les indicatrices de manquant pour détecter des structures.

```

1 var.na <- which(res.aggr$missing$Count > 0)
2 pattern <- is.na(diabetes[, var.na])
3 matcram <- PairApply(pattern, CramerV)
4 PlotCorr(matcram)

```

Analyse multivariée : une ACM sur le pattern de manquance.

```

1 MCA(pattern)

```

Nature du mécanisme de manquance

On explore les liens entre valeurs manquantes et observées :

```

1 # Une variable complete (Age) et une incomplete (Skin.Thick)
2 marginmatrix(diabetes[, c("Age", "Skin.Thick")])
3 # Rouge = individus avec valeur manquante sur Skin.Thick
4 # Les individus incomplets semblent plus ages
5
6 # Tous les couples
7 marginmatrix(diabetes[, -ncol(diabetes)], cex = .2, gap = 0)
8
9 # Matrixplot
10 matrixplot(diabetes, sortby = 8)

```

À retenir. Si les individus avec valeur manquante sur une variable ont des distributions différentes sur une autre variable (complète), le mécanisme est probablement MAR (Missing At Random).

ACM sur données discrétisées :

```

1 d_bins <- discretize_get_bins(data = diabetes)
2 diabetes.cat <- discretize_df(diabetes, d_bins)
3 res.mca <- MCA(diabetes.cat, graph = FALSE,
4               level.ventil = 0.04)
5 plot(res.mca, choix = "ind", invisible = "ind")

```

Imputation multiple avec MICE

Paramètres par défaut

La fonction `mice` du package `mice` utilise par défaut :

- `m` = 5 tableaux imputés.
- `maxit` = 5 itérations.
- Méthodes par défaut : `pmm` (predictive mean matching) pour les variables numériques, `logreg` pour les binaires, `polyreg` pour les multinomiales.

On lance MICE avec `m` = 5 et `maxit` = 50 :

```

1 res.mice <- mice(diabetes, maxit = 50, printFlag = FALSE)
2 tableau1 <- complete(res.mice, 1)
3 summary(tableau1)

```

Vérification de la convergence

```

1 plot(res.mice)

```

Les courbes de convergence doivent montrer une stabilisation des paramètres au fil des itérations (pas de tendance monotone).

Comparaison des distributions imputées vs observées

```
1 densityplot(res.mice)
```

Les distributions des valeurs imputées (en rouge) doivent ressembler à celles des valeurs observées (en bleu) si le modèle d'imputation est correct.

Analyse par marginmatrix

On compare les couples de variables avant et après imputation :

```
1 diabetes.vim <- cbind.data.frame(complete(res.mice),
2                                 is.na(diabetes))
3 colnames(diabetes.vim) <- c(colnames(diabetes),
4                             paste0(colnames(diabetes), "_imp"))
5
6 # Couple Age - Skin.Thick
7 par(mfrow = c(1, 2))
8 marginmatrix(diabetes[, c("Age", "Skin.Thick")])
9 marginmatrix(diabetes.vim[, c("Age", "Skin.Thick",
10                               "Age_imp", "Skin.Thick_imp")], delimiter = "_imp")
11
12 # Tous les couples
13 marginmatrix(diabetes.vim, delimiter = "_imp")
```

Les points imputés (en marron/jaune) doivent se situer dans le nuage des points observés.

Overimputation

L'overimputation consiste à traiter chaque valeur observée comme manquante, l'imputer, et comparer l'imputation à la valeur réelle.

```
1 nnodes <- detectCores() - 1
2
3 # Imputation avec m = 100 pour overimputation
4 res.mice.over <- mice.par(diabetes, m = 100,
5                           nnodes = nnodes, maxit = 20)
6
7 # Overimputation sur BMI (variable 6)
8 plotinds <- sample(seq(nrow(diabetes)), size = 30)
9 res.over <- overimpute(res.mice.over,
10                        plotinds = plotinds,
11                        plotvar = 6,
12                        nnodes = nnodes)
```

À retenir. L'overimputation est un outil de diagnostic : si les valeurs imputées sont systématiquement éloignées des valeurs observées, le modèle d'imputation est inadapté.

Choix des méthodes d'imputation

Méthodes disponibles pour MICE

On examine les méthodes par défaut et on teste des alternatives pour BMI :

```
1 res.mice$method # methodes par defaut
2 # On teste trois methodes pour BMI
3 par(mfrow = c(2, 2), mar = c(4, 4, 2, 1) + 0.1)
4 for (method.imp in c("rf", "norm", "norm.boot")){
5   method <- res.mice$method
6   method["BMI"] <- method.imp
```

```

7  res.mice.tmp <- mice(diabetes, method = method,
8    maxit = 20, printFlag = FALSE)
9  print(densityplot(res.mice.tmp, ~ BMI, main = method.imp))
10 }
```

Méthodes testées :

- `pmm` (défaut) : Predictive Mean Matching.
- `norm` : régression linéaire bayésienne.
- `norm.boot` : régression linéaire avec bootstrap.
- `rf` : Forêt aléatoire.

Comparaison par overimputation

On compare `norm` et `rf` via overimputation :

```

1  # norm
2  method <- res.mice$method
3  method["BMI"] <- "norm"
4  res.mice.over.norm <- mice.par(diabetes, m = 100,
5    nnodes = nnodes, maxit = 20, method = method)
6  res.over.norm <- overimpute(res.mice.over.norm,
7    plotinds = plotinds, plotvars = 6, nnodes = nnodes)
8
9  # rf
10 method["BMI"] <- "rf"
11 res.mice.over.rf <- mice.par(diabetes, m = 100,
12   nnodes = nnodes, maxit = 20, method = method)
13 res.over.rf <- overimpute(res.mice.over.rf,
14   plotinds = plotinds, plotvars = 6, nnodes = nnodes)
```

Analyse après imputation

Régression logistique sur chaque tableau imputé

On ajuste un modèle de régression logistique sur chaque tableau imputé avec `with.mids` :

```

1  fit <- with(res.mice,
2    glm(Outcome ~ Preg + Glucose + BP + Skin.Thick
3      + Insulin + BMI + DPF + Age, family = binomial))
4  length(fit$analyses) # 5 modeles (un par tableau impute)
```

Pooling des résultats (règles de Rubin)

```

1  res.pool <- pool(fit)
2  summary(res.pool)
```

Le **pooling** combine les estimations des m modèles en utilisant les règles de Rubin :

- $\bar{\theta} = \frac{1}{m} \sum_{k=1}^m \hat{\theta}_k$ (moyenne des estimations).
- Variance totale : $T = W + (1 + \frac{1}{m})B$, où W est la variance intra-imputation et B la variance inter-imputation.

Comparaison avec les cas-complets

```
1 res.glm.cc <- glm(Outcome ~ ., family = binomial,
2   data = diabetes)
3 summary(res.glm.cc)
```

On compare les coefficients et les écarts-types entre l'analyse sur cas-complets et l'analyse après imputation multiple. Des différences importantes peuvent indiquer un biais de sélection dû aux données manquantes.

Analyse de sensibilité

L'analyse de sensibilité par la méthode du **delta** (*delta adjustment*) permet de tester la robustesse des résultats sous différents scénarios pour le mécanisme de manquant (notamment l'hypothèse MNAR).

On applique un décalage δ aux valeurs imputées de Skin.Thick :

```
1 boxplot(diabetes$Skin.Thick)
2 delta <- c(0, 10, 20, 50)
3 imp.all <- vector("list", length(delta))
4 names(imp.all) <- delta
5 post <- res.mice.over$post
6 for (i in 1:length(delta)){
7   d <- delta[i]
8   cmd <- paste("imp[[j]][,i] <- imp[[j]][,i] +", d)
9   post["Skin.Thick"] <- cmd
10  imp <- mice(diabetes, post = post, maxit = 5,
11    seed = i, print = FALSE)
12  imp.all[[i]] <- imp
13 }
```

On compare les distributions imputées pour $\delta = 0$ et $\delta = 50$:

```
1 par(mfrow = c(1, 2))
2 bwplot(imp.all[["0"]])
3 bwplot(imp.all[["50"]])
4 densityplot(imp.all[["0"]], lwd = 3)
5 densityplot(imp.all[["50"]], lwd = 3)
```

Si les conclusions de l'analyse changent peu malgré un décalage important, les résultats sont robustes à l'hypothèse MNAR.

Résumé du pipeline

1. **Import** : charger les données et les packages (mice, VIM, micemd, etc.).
2. **Analyse exploratoire** : étudier les liens entre variables (corrélations, boxplots).
3. **Analyse du pattern de manquant** : univariée (aggr), bivariée (V de Cramer), multivariée (ACM).
4. **Nature du mécanisme** : marginmatrix, matrixplot.
5. **Imputation multiple** : lancer mice() avec les paramètres choisis (m , maxit, méthodes).
6. **Diagnostics** : convergence (plot), distribution (densityplot), marginmatrix, overimputation.
7. **Analyse des tableaux imputés** : with.mids + pool.
8. **Comparaison avec cas-complets** : vérifier la présence de biais.
9. **Analyse de sensibilité** : delta adjustment pour tester l'hypothèse MNAR.

Questions clés (QCM)

1. Quel package R implémente l'imputation multiple par équations chaînées ?

- (a) VIM
- (b) mice
- (c) missMDA
- (d) Amelia

Réponse : b (package mice).

2. Quelle est la méthode d'imputation par défaut de `mice` pour les variables numériques ?

- (a) régression linéaire bayésienne
- (b) predictive mean matching (pmm)
- (c) forêt aléatoire
- (d) moyenne conditionnelle

Réponse : b (pmm).

3. Combien de tableaux imputés sont créés par défaut par `mice()` ?

- (a) 3
- (b) 5
- (c) 10
- (d) 20

Réponse : b ($m = 5$ par défaut).

4. À quoi sert la fonction `plot(res.mice)` ?

- (a) Visualiser la distribution des données
- (b) Vérifier la convergence de l'algorithme MICE
- (c) Afficher les valeurs manquantes
- (d) Comparer les imputations

Réponse : b (vérification de la convergence).

5. Quelle fonction permet de comparer les distributions des valeurs imputées et observées ?

- (a) `aggr`
- (b) `densityplot`
- (c) `marginmatrix`
- (d) both a and b

Réponse : d (`densityplot` et `marginmatrix`).

6. Comment appelle-t-on la technique consistant à traiter chaque valeur observée comme manquante pour évaluer le modèle d'imputation ?

- (a) Validation croisée
- (b) Bootstrap
- (c) Overimputation
- (d) Delta adjustment

Réponse : c (overimputation).

7. Quelle fonction permet d'agréger les résultats de m modèles ajustés sur les tableaux imputés ?

- (a) `with.mids`
- (b) `pool`
- (c) `complete`
- (d) `summary`

Réponse : b (pool).

8. Dans l'analyse de sensibilité par delta adjustment, que représente δ ?

- (a) Le nombre d'itérations supplémentaires
- (b) Le décalage ajouté aux valeurs imputées
- (c) Le seuil de détection des atypiques
- (d) Le taux de valeurs manquantes

Réponse : b (décalage δ ajouté aux valeurs imputées pour tester MNAR).

9. L'analyse de sensibilité par delta adjustment permet de tester l'hypothèse :

- (a) MCAR
- (b) MAR
- (c) MNAR
- (d) Aucune

Réponse : c (MNAR).

10. Quel est l'avantage principal de l'imputation multiple par rapport à l'imputation simple ?

- (a) Elle est plus rapide
- (b) Elle prend en compte l'incertitude due aux valeurs manquantes
- (c) Elle ne nécessite pas de modèle
- (d) Elle ne produit qu'un seul tableau

Réponse : b (prise en compte de l'incertitude via la variance inter-imputation).

Références

- [1] V. Audigier. *Application mice*. Cours CNAM, 2025.
- [2] S. van Buuren. *Flexible Imputation of Missing Data*. 2nd ed., CRC Press, 2018.
- [3] S. van Buuren, K. Groothuis-Oudshoorn. mice : Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software*, 45(3), 2011.
- [4] V. Audigier, F. Husson. *micemd : Multiple Imputation by Chained Equations with Multilevel Data*. <https://CRAN.R-project.org/package=micemd>